Python 编程 · 第 4 章 学习包

可迭代对象

序列、字典和集合

列表 list · 元组 tuple · 字典 dict · 集合 set · *args / **kwargs

李相君 · 2026软工(NIIT)3班 · 2026年10月29日

目录

第0部分　先看清楚：这一章到底在讲什么
第1部分　考试怎么考 & 哪些是重点
第2部分　列表 list（本章最核心）
第3部分　元组 tuple
第4部分　元组和列表的区别（必考简答）
第5部分　字典 dict（第二重点）
第6部分　集合 set（课本标题里的“集合”）
第7部分　`*args` 和 `**kwargs`（理解即可）
第8部分　易错点汇总（考前必看）
第9部分　练习题
第10部分　练习答案与讲解
第11部分　一页速查表

所有示例代码均在 Python 3.13 中实际运行，“运行结果”为真实输出。

第0部分　先看清楚：这一章到底在讲什么

> **一句话概括：** 这一章教你用 4 种“容器”把一堆数据装在一起——**列表 list、元组 tuple、字典 dict、集合 set**，以及怎么对它们增、删、改、查。

前面你学的变量，一个变量只能装一个值，比如 `score = 90`。可现实中数据经常是“一堆”：全班 40 个人的成绩、播放列表里的歌、游戏背包里的道具……如果每个都单独起一个变量名（`score1`、`score2`……`score40`），会累死。**容器就是一个能装很多值的“盒子”**，你只要记住盒子的名字就行。

容器	长什么样	一句话特点	生活类比
列表 list	`[1, 2, 3]` 方括号	**有顺序、能修改**、可以重复	购物清单，可以随时加、划掉、改
元组 tuple	`(1, 2, 3)` 圆括号	**有顺序、不能修改**	刻在石碑上的字，写完就定死了
字典 dict	`{"name": "小明"}` 花括号+冒号	**用“键”找“值”**，键不能重复	通讯录：用名字查电话
集合 set	`{1, 2, 3}` 花括号	**没顺序、自动去重**	一个袋子，同样的东西只留一个

术语小词典（课本里会反复出现，先混个脸熟）：

- **元素 / 项（item）**：容器里装的每一个值。
- **序列（sequence）**：元素排好顺序、能用“第几个”来取的容器。列表、元组、字符串都是序列。（课本小结里的“定义顺序”应该就是“定义序列”。）
- **可迭代对象（iterable）**：能用 `for` 循环一个一个取出来的东西。本章 4 种容器全都是。
- **可变（mutable）/ 不可变（immutable）**：创建以后能不能改里面的内容。列表、字典、集合可变；元组不可变。

代码

```
scores = [90, 85, 77, 92]
print(scores)
print(type(scores))
print(len(scores))
```

运行结果

```
[90, 85, 77, 92]
<class 'list'>
4
```

“可迭代”最直观的体现就是能被 `for` 循环遍历：

代码

```
fruits = ["苹果", "香蕉", "橘子"]
for f in fruits:
    print("我喜欢吃", f)
```

运行结果

```
我喜欢吃 苹果
我喜欢吃 香蕉
我喜欢吃 橘子
```

第1部分　考试怎么考 & 哪些是重点

> 说明：下面是 Python 入门课程里对这一章**最常见的考法**，具体以你们李老师的要求为准。如果老师划了重点或给了往年题，发给我，我再帮你对一遍。

常见题型：

1. **读代码写输出**（最常见）：给一段操作列表/字典的代码，问 `print` 出来是什么。考索引、切片、各种方法的效果。
2. **选择/判断题**：`append` 和 `extend` 有什么区别？`remove` 和 `pop` 有什么区别？元组能不能改？
3. **找错题**：改元组报 `TypeError`、取字典不存在的键报 `KeyError`、下标越界报 `IndexError`。
4. **编程题**：求列表最大/最小/平均值、统计每个字出现几次（字典）、做个简单通讯录/成绩单。

重点划分：

等级	内容	要求
★★★ 必须掌握	索引（含负索引）、切片 `[start:stop:step]`	看到就能说出结果
★★★ 必须掌握	列表方法：`append` `insert` `extend` `remove` `pop` `del` `sort`	会用、会区分、会写输出
★★★ 必须掌握	字典：创建、`[]` 取值、`get`、增改删、`items()` 遍历	能写编程题
★★ 要会	`index` `count` `reverse` `clear` `copy`、`len/max/min/sum`、`in`	会用
★★ 要会	元组的创建、索引切片、**不能修改**、与列表的区别	能答区别题
★ 理解即可	`*args` 和 `**kwargs`	知道它们分别收成元组和字典
★ 理解即可	集合去重、交并差	知道怎么用

建议学习顺序： 第2部分（列表）是整章的地基，占一半篇幅，一定先吃透；元组和集合是“列表的变种”，很快；字典是第二个重点；`*args/**kwargs` 放最后。

怎么用这份资料： 每段代码下面都有“运行结果”（全部在电脑上真实跑过）。**最好的学法是：先遮住运行结果，自己猜输出，再对答案。** 猜错的地方就是你要重点看的地方。能打开电脑的话，把代码亲手敲一遍，效果翻倍。

第2部分　列表 list（本章最核心）

2.1 创建列表

用方括号 `[]` 把元素括起来，元素之间用逗号隔开。一个列表里什么类型都能装，甚至能装另一个列表（叫“嵌套列表”）。

代码

```
empty = []
nums = [1, 2, 3]
mixed = [1, "hello", 3.14, True]
nested = [[1, 2], [3, 4]]
from_str = list("abc")
print(empty, nums, mixed, nested, from_str)
```

运行结果

```
[] [1, 2, 3] [1, 'hello', 3.14, True] [[1, 2], [3, 4]] ['a', 'b', 'c']
```

> `list("abc")` 会把字符串拆成一个个字符，变成列表。

2.2 索引：取出“第几个”

这是全章最重要的一个规则：Python 从 0 开始数！ 第 1 个元素的编号（下标/索引）是 0，第 2 个是 1……

还可以**倒着数**：`-1` 是最后一个，`-2` 是倒数第二个。

元素	"亚瑟"	"妲己"	"后羿"	"鲁班"
正索引	0	1	2	3
负索引	-4	-3	-2	-1

代码

```
heroes = ["亚瑟", "妲己", "后羿", "鲁班"]
print(heroes[0])
print(heroes[2])
print(heroes[-1])
print(heroes[-2])
```

运行结果

```
亚瑟
后羿
鲁班
后羿
```

如果编号超出范围，Python 会报错。4 个元素的列表，最大索引是 3，取 `[4]` 就越界了：

代码

```
heroes = ["亚瑟", "妲己", "后羿", "鲁班"]
print(heroes[4])
```

运行结果（报错）

```
IndexError: list index out of range
```

> **易错：** 列表有 n 个元素，最后一个的索引是 **n-1**，不是 n。

2.3 切片：一次取出“一段”

切片的格式是 `列表[start:stop:step]`，意思是“从 start 开始，到 stop **之前**停下，每次跨 step 步”。

> **切片三条口诀：**
>
> 1. **包头不包尾**（左闭右开）：`[1:4]` 取的是索引 1、2、3，**不含 4**。
> 2. **省略就是到头**：start 不写＝从开头；stop 不写＝到结尾。
> 3. **step 是步长**：`2` 就是隔一个取一个；`-1` 就是倒过来。

元素	10	20	30	40	50	60
正索引	0	1	2	3	4	5
负索引	-6	-5	-4	-3	-2	-1

代码

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[1:4])
print(nums[:3])
print(nums[3:])
print(nums[:])
print(nums[-3:])
print(nums[::2])
print(nums[1:5:2])
print(nums[::-1])
```

运行结果

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 20, 30, 40, 50, 60]
[40, 50, 60]
[10, 30, 50]
[20, 40]
[60, 50, 40, 30, 20, 10]
```

逐行解释：

- `nums[1:4]`：索引 1 到 3 → `[20, 30, 40]`
- `nums[:3]`：从头到索引 2 → 前 3 个

- `nums[3:]`：从索引 3 到最后
- `nums[:]`：全部（相当于复制了一份）
- `nums[-3:]`：从倒数第 3 个到最后 → **最后 3 个**
- `nums[::2]`：从头到尾，每隔一个取 → 索引 0、2、4
- `nums[1:5:2]`：索引 1 到 4，每隔一个 → 索引 1、3
- `nums[::-1]`：**整个倒过来**（考试特别爱考）

> 小技巧：`[a:b]` 取出的元素个数正好是 **b - a** 个。`[1:4]` 就是 3 个。

2.4 修改某个元素

列表是可变的，直接给某个位置重新赋值就行：

代码

```
team = ["亚瑟", "妲己", "后羿"]
team[1] = "王昭君"
print(team)
```

运行结果

```
['亚瑟', '王昭君', '后羿']
```

2.5 增加元素：append / insert / extend

方法	作用	记忆法
`append(x)`	把 x **整个**加到**末尾**	append＝“追加”，排到队尾
`insert(i, x)`	把 x 插到**索引 i 的位置**，后面的往后挪	insert＝“插队”
`extend(列表)`	把另一个列表的元素**拆开**，一个个加到末尾	extend＝“扩展”，两队合并

append：加到最后

代码

```
bag = ["剑"]
bag.append("盾")
print(bag)
bag.append(["药水", "卷轴"])
print(bag)
print(len(bag))
```

运行结果

```
['剑', '盾']
['剑', '盾', ['药水', '卷轴']]
3
```

注意第二次 `append` 加进去的是一个**整体**（一个列表），所以 `bag` 只有 3 个元素，最后一个元素本身是个列表。

insert：插到指定位置

代码

```
queue = ["小王", "小李", "小张"]
queue.insert(1, "小红")
print(queue)
queue.insert(0, "队长")
print(queue)
```

运行结果

```
['小王', '小红', '小李', '小张']
['队长', '小王', '小红', '小李', '小张']
```

`insert(1, "小红")` 的意思是“让小红站到索引 1 的位置”，原来索引 1 及之后的人都往后挪一位。

extend vs append（必考区别）

代码

```
a = [1, 2]
a.extend([3, 4])
print(a)
b = [1, 2]
b.append([3, 4])
print(b)
```

运行结果

```
[1, 2, 3, 4]
[1, 2, [3, 4]]
```

> **一句话区分：** `append` 把整个东西当**一个**元素塞进去；`extend` 把东西**拆开**一个个塞进去。

2.6 删除元素：remove / pop / del / clear

课本小结里把这一组放在“从列表中删除项目”下面（课本把 append 也列在这里，其实它是添加，不用管）。删除有四种方式，**区别必考**：

方式	按什么删	有返回值吗	例子
`remove(x)`	按**值**删，只删**第一个**匹配的	没有	`pets.remove("猫")`
`pop()`	按**位置**删，不写默认删**最后一个**	**有，返回被删的那个**	`last = stack.pop()`
`pop(i)`	删索引 i 的元素	有	`stack.pop(1)`
`del 列表[i]`	按位置删，还能删一段切片	没有（它是语句，不是方法）	`del nums[0]`
`clear()`	全部清空，列表还在但变空了	没有	`todo.clear()`

remove：按值删

代码

```
pets = ["猫", "狗", "猫", "鱼"]
pets.remove("猫")
print(pets)
```

运行结果

```
['狗', '猫', '鱼']
```

列表里有两个“猫”，`remove` 只删了**第一个**。如果要删的值根本不存在，会报错：

代码

```
pets = ["猫", "狗"]
pets.remove("兔子")
```

运行结果（报错）

```
ValueError: list.remove(x): x not in list
```

pop：按位置删，并把删掉的东西“交给你”

代码

```
stack = ["a", "b", "c", "d"]
last = stack.pop()
print(last, stack)
second = stack.pop(1)
print(second, stack)
```

运行结果

```
d ['a', 'b', 'c']
b ['a', 'c']
```

`pop` 像从一摞书里抽走一本拿在手上——你能拿到被抽走的那本（返回值），而 `remove`、`del` 删了就没了。

del：按位置删（还能删一段）

代码

```
nums = [10, 20, 30, 40, 50]
del nums[0]
print(nums)
del nums[1:3]
print(nums)
```

运行结果

```
[20, 30, 40, 50]
[20, 50]
```

第二行 `del nums[1:3]` 删除的是当时列表 `[20, 30, 40, 50]` 中索引 1、2 的元素（30 和 40）。

clear：一键清空

代码

```
todo = ["写作业", "洗衣服"]
todo.clear()
print(todo)
```

运行结果

```
[]
```

2.7 查找和统计：index / count / in

- `index(x)`：返回 x **第一次出现**的索引（找不到会报错）。
- `count(x)`：数 x 出现了几次。
- `x in 列表`：判断 x 在不在列表里，返回 `True` / `False`。这个不是方法，但非常常用。

代码

```
letters = ["a", "b", "c", "b", "b"]
print(letters.index("b"))
print(letters.count("b"))
print("c" in letters)
print("z" in letters)
```

运行结果

```
1
3
True
False
```

2.8 排序和反转：sort / reverse

`sort()` 把列表**从小到大**排好；加上 `reverse=True` 就是从大到小。字符串按字母顺序排。

代码

```
nums = [5, 2, 9, 1]
nums.sort()
print(nums)
nums.sort(reverse=True)
print(nums)
words = ["banana", "apple", "cherry"]
words.sort()
print(words)
```

运行结果

```
[1, 2, 5, 9]
[9, 5, 2, 1]
['apple', 'banana', 'cherry']
```

超级易错： `sort()` 是**直接改原列表**，它本身返回 `None`（“什么都没有”）。所以千万别写 `result = nums.sort()`：

代码

```
nums = [3, 1, 2]
result = nums.sort()
print(result)
print(nums)
```

运行结果

```
None
[1, 2, 3]
```

如果你想**保留原列表**、得到一个排好序的新列表，用内置函数 `sorted()`：

代码

```
nums = [3, 1, 2]
new = sorted(nums)
print(new)
print(nums)
```

运行结果

```
[1, 2, 3]
[3, 1, 2]
```

`reverse()` 把列表**倒过来**（注意：不是排序，只是把顺序反转）：

代码

```
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)
```

运行结果

```
[4, 3, 2, 1]
```

> `reverse()` 改原列表；`[::-1]` 生成一个倒过来的新列表，原列表不变。

2.9 复制：copy（理解“同一个盒子”）

这是很多同学想不通的地方，用一个比喻：变量名就像**贴在盒子上的标签**。

- `b = a`：没有新盒子！只是在**同一个盒子**上又贴了一张标签 b。改 a 就是改 b。
- `c = a.copy()`：复制出**一个新盒子** c，之后 a 怎么变都和 c 无关。

代码

```
a = [1, 2, 3]
b = a
c = a.copy()
a.append(4)
print("a =", a)
print("b =", b)
print("c =", c)
```

运行结果

```
a = [1, 2, 3, 4]
b = [1, 2, 3, 4]
c = [1, 2, 3]
```

2.10 好用的内置函数和运算

代码

```
nums = [3, 8, 1, 6]
print(len(nums), max(nums), min(nums), sum(nums))
print([1, 2] + [3])
print(["哈"] * 3)
```

运行结果

```
4 8 1 18
[1, 2, 3]
['哈', '哈', '哈']
```

- `len()` 个数，`max()` 最大，`min()` 最小，`sum()` 求和
- `+` 把两个列表拼起来，`*` 把列表重复几次

第3部分　元组 tuple

3.1 创建元组

元组和列表几乎一样，区别是用**圆括号**，而且**创建后不能改**。其实圆括号可以省略，真正起作用的是逗号。

代码

```
point = (3, 5)
colors = ("红", "绿", "蓝")
t = 1, 2, 3
print(point, colors, t)
print(type(t))
```

运行结果

```
(3, 5) ('红', '绿', '蓝') (1, 2, 3)
<class 'tuple'>
```

> **易错：只有一个元素的元组，必须加逗号！** `(5)` 只是数字 5 加了个括号；`(5,)` 才是元组。

代码

```
a = (5)
b = (5,)
print(type(a))
print(type(b))
```

运行结果

```
<class 'int'>
<class 'tuple'>
```

3.2 索引元组、切片元组

规则**和列表完全一样**（课本小结里的“索引元组”“切片元组”就是这个）。元组也有 `count` 和 `index` 两个方法。

代码

```
week = ("周一", "周二", "周三", "周四", "周五")
print(week[0])
print(week[-1])
print(week[1:3])
print(week.count("周二"), week.index("周三"))
```

运行结果

```
周一
周五
('周二', '周三')
1 2
```

3.3 元组不能修改

代码

```
week = ("周一", "周二", "周三")
week[0] = "星期一"
```

运行结果（报错）

```
TypeError: 'tuple' object does not support item assignment
```

所以元组**没有** `append`、`remove`、`sort` 这些会改变内容的方法。

3.4 元组的常见用法：解包

可以把元组里的值一次性“拆”给几个变量，Python 交换两个变量就是靠这个：

代码

```
x, y = (3, 5)
print(x, y)
a, b = 1, 2
a, b = b, a
print(a, b)
```

运行结果

```
3 5
2 1
```

列表和元组可以互相转换：

代码

```
t = (1, 2, 3)
lst = list(t)
lst.append(4)
t2 = tuple(lst)
print(lst, t2)
```

运行结果

```
[1, 2, 3, 4] (1, 2, 3, 4)
```

第4部分　元组和列表的区别（必考简答）

对比项	列表 list	元组 tuple
符号	方括号 `[ ]`	圆括号 `( )`
能否修改	**可变**：能增、删、改	**不可变**：创建后不能改
方法	很多：append、insert、remove、pop、sort……	只有 `count()` 和 `index()`
共同点	都有序、都能索引和切片、都能 for 遍历、都能重复	同左
速度/安全	灵活	更省内存、更快；数据不会被误改
适合装	会变化的数据（购物车、成绩单）	固定不变的数据（坐标、星期、颜色 RGB）
能当字典的键吗	不能	能（因为不可变）

考试答题模板： “列表和元组都是有序序列，都支持索引和切片。区别在于列表是可变的，可以增删改元素，用方括号表示；元组是不可变的，创建后不能修改，用圆括号表示，因此更安全、占用资源更少，适合存放不需要修改的数据。”

第5部分　字典 dict（第二重点）

5.1 什么是“键值对”

列表是用“第几个”来找东西；字典是用“**名字**”来找东西。字典里每一项由两部分组成：**键（key）：值（value）**。就像通讯录：键是名字，值是电话。

> **字典三条规则：**
>
> 1. 写法：`{键1: 值1, 键2: 值2}`
> 2. **键不能重复**（重复的话后面的覆盖前面的），键一般用字符串或数字
> 3. 值可以是任何东西，可以重复

代码

```
student = {"name": "小明", "age": 18, "major": "软件工程"}
print(student)
print(len(student))
empty = {}
print(type(empty))
```

运行结果

```
{'name': '小明', 'age': 18, 'major': '软件工程'}
3
<class 'dict'>
```

> 注意：空的 `{}` 是**空字典**，不是空集合。

5.2 取值：[] 和 get()

- `字典[键]`：直接取。**键不存在会报 KeyError 错误。**
- `字典.get(键)`：安全地取。键不存在时返回 `None`，或返回你指定的默认值，**不会报错**。

代码

```
student = {"name": "小明", "age": 18}
print(student["name"])
print(student.get("age"))
print(student.get("phone"))
print(student.get("phone", "未填写"))
```

运行结果

```
小明
18
None
未填写
```

代码

```
student = {"name": "小明", "age": 18}
print(student["phone"])
```

运行结果（报错）

```
KeyError: 'phone'
```

> **做题建议：** 不确定键在不在的时候，用 `get(键, 默认值)`，最安全。

5.3 增加和修改：同一个写法

`字典[键] = 值`：键**已存在**就是**修改**，键**不存在**就是**新增**。

代码

```
student = {"name": "小明", "age": 18}
student["age"] = 19
student["city"] = "海口"
print(student)
```

运行结果

```
{'name': '小明', 'age': 19, 'city': '海口'}
```

还可以用 `update()` 一次合并另一个字典（有的改、没有的加）：

代码

```
a = {"x": 1, "y": 2}
a.update({"y": 20, "z": 30})
print(a)
```

运行结果

```
{'x': 1, 'y': 20, 'z': 30}
```

5.4 删除：pop() 和 del

代码

```
student = {"name": "小明", "age": 19, "city": "海口"}
city = student.pop("city")
print(city, student)
del student["age"]
print(student)
```

运行结果

```
海口 {'name': '小明', 'age': 19}
{'name': '小明'}
```

和列表类似：`pop(键)` 会把被删掉的值返回给你；`del` 删了就没了。

5.5 遍历字典：keys / values / items（编程题必用）

- `keys()`：所有的键
- `values()`：所有的值
- `items()`：所有的（键, 值）对——**最常用**，配合 for 一次拿到两个
- `键 in 字典`：判断的是**键**在不在，不是值

代码

```
scores = {"语文": 88, "数学": 95, "英语": 79}
print(scores.keys())
print(scores.values())
print(scores.items())
for subject, score in scores.items():
    print(subject, "考了", score, "分")
print("数学" in scores)
print(95 in scores)
```

运行结果

```
dict_keys(['语文', '数学', '英语'])
dict_values([88, 95, 79])
dict_items([('语文', 88), ('数学', 95), ('英语', 79)])
语文 考了 88 分
数学 考了 95 分
英语 考了 79 分
True
False
```

注意最后一行：`95` 是值，不是键，所以 `95 in scores` 是 `False`。

5.6 经典例题：统计每个字母出现几次

这是字典最经典的编程题，**一定要会默写**：

代码

```
text = "banana"
count = {}
for ch in text:
    count[ch] = count.get(ch, 0) + 1
print(count)
```

运行结果

```
{'b': 1, 'a': 3, 'n': 2}
```

思路：遍历每个字符，`count.get(ch, 0)` 取出“目前已经数了几次”（第一次出现时没有这个键，就当 0），然后 +1 存回去。

第6部分　集合 set（课本标题里的“集合”）

集合就像一个**没有顺序、不能有重复**的袋子。最大的用处是**去重**和**求交集、并集**。

代码

```
s = {3, 1, 2, 3, 1}
print(s)
nums = [1, 2, 2, 3, 3, 3]
print(set(nums))
s.add(5)
s.remove(1)
print(s)
empty = set()
print(type(empty), type({}))
```

运行结果

```
{1, 2, 3}
{1, 2, 3}
{2, 3, 5}
<class 'set'> <class 'dict'>
```

- 重复的元素会被**自动去掉**
- 集合**没有顺序**，所以**不能用索引**（没有 `s[0]`）
- `add()` 添加，`remove()` 删除
- **空集合必须写 `set()`**，因为 `{}` 已经被字典占用了

集合运算（和高中数学的集合一样）：

代码

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b)
print(a | b)
print(a - b)
print(a ^ b)
```

运行结果

```
{3, 4}
{1, 2, 3, 4, 5}
{1, 2}
{1, 2, 5}
```

符号	含义	数学名称
`a & b`	两边都有的	交集
`a | b`	合在一起（去重）	并集
`a - b`	a 有但 b 没有的	差集
`a ^ b`	只在一边出现的	对称差

第7部分　`*args` 和 `**kwargs`（理解即可）

这两个是用在**函数定义**里的，作用是：**让函数能接收“不确定个数”的参数。**

7.1 `*args`：收集多余的位置参数 → 打包成元组

比如你想写一个求和函数，有时传 2 个数，有时传 4 个数：

代码

```
def total(*args):
    print("args =", args, type(args))
    return sum(args)

print(total(1, 2))
print(total(1, 2, 3, 4))
```

运行结果

```
args = (1, 2) <class 'tuple'>
3
args = (1, 2, 3, 4) <class 'tuple'>
10
```

前面的 `*` 是关键，意思是“把传进来的所有参数打包成一个元组，起名叫 args”。（名字 `args` 是习惯叫法，换成别的也行，`*` 不能省。）

7.2 `**kwargs`：收集多余的关键字参数 → 打包成字典

“关键字参数”就是调用时写成 `名字=值` 的形式：

代码

```
def show_info(**kwargs):
    print("kwargs =", kwargs)
    for key, value in kwargs.items():
        print(key, ":", value)

show_info(name="小明", age=18, city="海口")
```

运行结果

```
kwargs = {'name': '小明', 'age': 18, 'city': '海口'}
name : 小明
age : 18
city : 海口
```

两个 `**` 的意思是“把所有 `名字=值` 打包成一个字典，起名叫 kwargs”。（kw＝keyword 关键字。）

7.3 一起用

顺序必须是：**普通参数** → `*args` → `**kwargs`。

代码

```
def demo(a, *args, **kwargs):
    print("a =", a)
    print("args =", args)
    print("kwargs =", kwargs)

demo(1, 2, 3, x=10, y=20)
```

运行结果

```
a = 1
args = (2, 3)
kwargs = {'x': 10, 'y': 20}
```

7.4 反过来：调用时用 `*` 和 `**` 拆包（了解）

代码

```
def add3(a, b, c):
    return a + b + c

nums = [1, 2, 3]
print(add3(*nums))
d = {"a": 10, "b": 20, "c": 30}
print(add3(**d))
```

运行结果

```
6
60
```

> **一张图记住：** 单星 `*` ↔ 元组（一串值）；双星 `**` ↔ 字典（名字=值）。定义函数时是“打包”，调用函数时是“拆包”。

第8部分　易错点汇总（考前必看）

编号	易错点	正确理解
1	第一个元素是 `[1]`	**从 0 开始**，第一个是 `[0]`，最后一个是 `[-1]`
2	`[1:4]` 包含索引 4	**包头不包尾**，只有 1、2、3
3	`append` 和 `extend` 一样	`append` 整体放进去；`extend` 拆开放
4	`remove` 删掉所有匹配项	只删**第一个**；不存在会报 ValueError
5	`result = nums.sort()`	`sort()` 返回 None；要新列表用 `sorted()`
6	`b = a` 就是复制	只是贴了个新标签，同一个列表；复制用 `a.copy()`
7	`(5)` 是元组	单元素元组要写 `(5,)`
8	元组可以改元素	不能，会报 TypeError
9	`字典[不存在的键]` 会返回空	会报 KeyError；安全取值用 `get()`
10	`值 in 字典` 判断值	`in` 判断的是**键**
11	`{}` 是空集合	`{}` 是空字典，空集合是 `set()`
12	集合可以用 `s[0]`	集合没有顺序，不能索引

第9部分　练习题

> 先自己做，做完再翻到第10部分对答案。前 6 题是“读代码写输出”，后 4 题是编程题。

练习1（索引与切片） 写出每行的输出：

代码

```
a = [1, 2, 3, 4, 5, 6]
print(a[2])
print(a[-2])
print(a[1:4])
print(a[::-1])
print(a[::3])
```

练习2（append vs extend） 两次 `len` 分别输出多少？

代码

```
x = ["a", "b"]
x.append(["c", "d"])
print(len(x))
y = ["a", "b"]
y.extend(["c", "d"])
print(len(y))
```

练习3（删除与插入） 写出每行输出：

代码

```
nums = [5, 3, 8, 3]
nums.remove(3)
print(nums)
p = nums.pop()
print(p, nums)
nums.insert(0, 9)
print(nums)
```

练习4（复制陷阱） 输出是什么？

代码

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

练习5（字典） 写出每行输出：

代码

```
d = {"苹果": 5, "香蕉": 3}
d["橘子"] = 4
d["苹果"] = 6
print(d)
print(d.get("西瓜", 0))
print(len(d))
```

练习6（`*args` / `**kwargs`） 输出是什么？

代码

```
def f(*args, **kwargs):
    print(len(args), len(kwargs))
f(1, 2, 3, a=4)
```

编程题1： 有成绩列表 `scores = [78, 92, 65, 88, 99, 54]`，输出最高分、最低分、平均分（保留 1 位小数），并把及格（≥60）的成绩放进一个新列表输出。

编程题2： 有列表 `words = ["猫", "狗", "猫", "鱼", "狗", "猫"]`，用字典统计每个词出现的次数。

编程题3： 写一个函数 `average(*args)`，能接收任意个数字并返回平均值；如果一个数字都没传，返回 0。

编程题4： 用字典做一个简易通讯录：添加两个人，按名字查电话（查不到时输出“查无此人”），最后把所有人遍历输出。

第10部分　练习答案与讲解

练习1：

运行结果

```
3
5
[2, 3, 4]
[6, 5, 4, 3, 2, 1]
[1, 4]
```

`a[2]` 是第 3 个元素 3；`a[-2]` 是倒数第二个 5；`a[1:4]` 取索引 1~3；`[::-1]` 倒序；`[::3]` 取索引 0、3。

练习2：

运行结果

```
3
4
```

`append` 把 `["c","d"]` 当一个元素放进去，共 3 个；`extend` 拆开放，共 4 个。

练习3：

运行结果

```
[5, 8, 3]
3 [5, 8]
[9, 5, 8]
```

`remove(3)` 只删第一个 3；`pop()` 删掉并返回最后一个（3）；`insert(0, 9)` 把 9 放到最前面。

练习4：

运行结果

```
[1, 2, 3]
```

`b = a` 没有复制，a 和 b 是同一个列表，改 b 就是改 a。

练习5：

运行结果

```
{'苹果': 6, '香蕉': 3, '橘子': 4}
0
3
```

“橘子”不存在→新增；“苹果”已存在→修改为 6；“西瓜”不存在，`get` 返回默认值 0；一共 3 个键。

练习6：

运行结果

```
3 1
```

1、2、3 是位置参数，被 `*args` 收成 `(1, 2, 3)`，长度 3；`a=4` 被 `**kwargs` 收成 `{'a': 4}`，长度 1。

编程题1 参考答案：

代码

```
scores = [78, 92, 65, 88, 99, 54]
print("最高分:", max(scores))
print("最低分:", min(scores))
print("平均分:", round(sum(scores) / len(scores), 1))
passed = []
for s in scores:
    if s >= 60:
        passed.append(s)
print("及格的:", passed)
```

运行结果

```
最高分: 99
最低分: 54
平均分: 79.3
及格的: [78, 92, 65, 88, 99]
```

编程题2 参考答案：

代码

```
words = ["猫", "狗", "猫", "鱼", "狗", "猫"]
count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
print(count)
```

运行结果

```
{'猫': 3, '狗': 2, '鱼': 1}
```

编程题3 参考答案：

代码

```
def average(*args):
    if len(args) == 0:
        return 0
    return sum(args) / len(args)

print(average(80, 90, 100))
print(average())
```

运行结果

```
90.0
0
```

编程题4 参考答案：

代码

```
book = {}
book["小明"] = "13800000001"
book["小红"] = "13800000002"
name = "小明"
print(name, "的电话是", book.get(name, "查无此人"))
print("小刚", "的电话是", book.get("小刚", "查无此人"))
for n, tel in book.items():
    print(n, tel)
```

运行结果

```
小明 的电话是 13800000001
小刚 的电话是 查无此人
小明 13800000001
小红 13800000002
```

第11部分　一页速查表

操作	列表 list	元组 tuple	字典 dict	集合 set
创建	`[1, 2]`	`(1, 2)` / `(1,)`	`{"a": 1}`	`{1, 2}` / `set()`
取值	`lst[0]`、`lst[1:3]`	`t[0]`、`t[1:3]`	`d["a"]`、`d.get("a")`	不能索引
添加	`append` `insert` `extend`	不能	`d["新键"] = 值`、`update`	`add`
删除	`remove` `pop` `del` `clear`	不能	`pop(键)` `del d[键]` `clear`	`remove`
修改	`lst[0] = 新值`	不能	`d["键"] = 新值`	—
查找	`in` `index` `count`	`in` `index` `count`	`键 in d`	`in`
遍历	`for x in lst`	`for x in t`	`for k, v in d.items()`	`for x in s`
有顺序?	有	有	按加入顺序	无
可重复?	可以	可以	键不能重复	不能
可修改?	可以	**不可以**	可以	可以